

4.6 XGBoost(eXtreme Gradient Boosting)

목차

1. XGBoost 개요
 1. XGBoost란
 2. 주요 장점
 3. 두 가지 파이썬 API
 4. 버전 확인
2. 파이썬 래퍼 XGBoost 적용 - 위스콘신 유방암 예측
 1. 데이터 로딩과 확인
 2. 학습·검증·테스트 데이터 분리
 3. DMatrix 생성
 4. 하이퍼 파라미터 설정
 5. train()으로 학습과 조기 중단
 6. predict()와 예측 확률
 7. 평가 함수와 성능 측정
 8. 피쳐 중요도 시각화
3. 사이킷런 래퍼 XGBoost
 1. XGBClassifier 기본 학습
 2. 조기 중단 적용
 3. early_stopping_rounds 값에 따른 차이
 4. 피쳐 중요도
4. 핵심 요약 및 머신러닝 활용 포인트

1. XGBoost 개요

1.1 XGBoost란

XGBoost(eXtreme Gradient Boosting)는 트리 기반 앙상블 학습에서 가장 각광받는 알고리즘 중 하나입니다. 캐글 경연 대회에서 상위권을 휩쓸며 유명해졌으며, GBM(Gradient Boosting Machine)에 기반하되 GBM의 느린 수행 시간과 과적합 규제 부재라는 단점을 해결한 것이 핵심입니다.

1.2 주요 장점

장점	설명
뛰어난 예측 성능	일반적으로 분류·회귀 영역에서 뛰어난 예측 성능을 발휘합니다.
GBM 대비 빠른 수행 시간	병렬 학습이 가능해 기존 GBM보다 훨씬 빠릅니다. 다만 다른 머신러닝 알고리즘(예: 랜덤 포레스트)에 비해 빠르다는 의미는 아닙니다.
과적합 규제	GBM에는 없는 규제(Regularization) 기능이 있어 과적합에 강합니다.
가지치기(Tree Pruning)	<code>max_depth</code> 로 분할 깊이를 조정하며, 더 이상 이득이 없는 분할을 가지치기해 분할 수를 줄입니다.
자체 내장 교차 검증	반복 수행 시마다 교차 검증을 수행해 최적화된 반복 횟수를 얻을 수 있습니다.
결손값 자체 처리	결손값을 자체적으로 처리할 수 있습니다.

■ 조기 중단(Early Stopping)

XGBoost의 대표적 기능입니다. 지정한 반복 횟수를 다 채우지 않더라도, 예측 오류가 더 이상 개선되지 않으면 중간에 학습을 멈춰 수행 시간을 단축합니다. 과적합 방지 효과도 있습니다. 본 장에서 직접 실습합니다.

1.3 두 가지 파이썬 API

구분	파이썬 래퍼 (네이티브)	사이킷런 래퍼
클래스/함수	<code>xgboost.train()</code>	<code>XGBClassifier</code> , <code>XGBRegressor</code>
데이터 형태	DMatrix 전용 객체	ndarray, DataFrame 그대로
하이퍼 파라미터	딕셔너리로 전달	생성자 인자로 전달
학습률 파라미터명	<code>eta</code>	<code>learning_rate</code>
반복 횟수	<code>num_boost_round</code>	<code>n_estimators</code>
predict 반환값	예측 확률	예측 레이블 (확률은 <code>predict_proba</code>)
사이킷런 연동	불가	<code>GridSearchCV</code> , <code>Pipeline</code> 등 그대로 사용

■ 어느 것을 써야 하나

실무에서는 사이킷런 래퍼를 권장합니다. 기존 사이킷런 코드와 호환되어 `cross_val_score`, `GridSearchCV` 등을 그대로 쓸 수 있기 때문입니다. 네이티브 API는 XGBoost 고유 기능을 세밀하게 제어할 때 사용합니다.

1.4 버전 확인

```
import xgboost

print(xgboost.__version__)
```

줄	코드	상세 설명
1	import xgboost	XGBoost는 사이킷런에 포함되지 않은 별도 패키지 이므로 <code>pip install xgboost</code> 로 설치해야 합니다.
3	print(xgboost.__version__)	버전을 확인합니다. 버전에 따라 API가 크게 달라지므로 반드시 확인하는 습관이 중요합니다(뒤의 경고 박스 참고).

▶ 실행 결과

```
3.2.0
```

2. 파이썬 래퍼 XGBoost 적용 - 위스콘신 유방암 예측

2.1 데이터 로딩과 확인

```
import xgboost as xgb
from xgboost import plot_importance
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

dataset = load_breast_cancer()
X_features= dataset.data
y_label = dataset.target

cancer_df = pd.DataFrame(data=X_features, columns=dataset.feature_names)
cancer_df['target']= y_label
cancer_df.head(3)
```

줄	코드	상세 설명
1	import xgboost as xgb	네이티브 API를 쓰기 위해 모듈 전체를 <code>xgb</code> 별칭으로 임포트합니다.
2	from xgboost import plot_importance	XGBoost가 제공하는 피쳐 중요도 시각화 함수 입니다. matplotlib 기반으로 막대 그래프를 그려 줍니다.
8	warnings.filterwarnings('ignore')	버전 관련 경고 메시지를 숨깁니다 . 출력을 깔끔하게 하지만, 중요한 API 변경 경고까지 가려질 수 있으므로 학습 단계에서는 주의가 필요합니다.
10~12	dataset = load_breast_cancer() 등	유방암 데이터를 로딩하고 피쳐(30개)와 레이블을 분리합니다.
14~15	cancer_df = pd.DataFrame(...); cancer_df['target']= y_label	피쳐를 DataFrame으로 만들고 레이블을 target 컬럼으로 추가 합니다. 이렇게 하면 뒤에서 <code>iloc</code> 슬라이싱으로 다시 분리하기 편합니다.

▶ 실행 결과 (앞 5개 컬럼만 표시)

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
0	17.99	10.38	122.8	1001.0	0.11840
1	20.57	17.77	132.9	1326.0	0.08474
2	19.69	21.25	130.0	1203.0	0.10960

```
print(dataset.target_names)
print(cancer_df['target'].value_counts())
```

줄	코드	상세 설명
1	print(dataset.target_names)	['malignant' 'benign'] → 0=악성(malignant), 1=양성(benign)입니다.
2	cancer_df['target'].value_counts()	양성 357건, 악성 212건입니다. 비율은 약 63% : 37%로 심하게 불균형하지는 않습니다.

▶ 실행 결과

```
['malignant' 'benign']
target
1    357
0    212
Name: count, dtype: int64
```

⚠ 레이블 의미에 주의

이 데이터에서 1이 양성(benign, 정상에 가까움), 0이 악성(malignant, 암)입니다. 일반적인 직관과 반대이므로 재현율(recall)을 해석할 때 주의해야 합니다. 뒤에서 계산하는 정밀도·재현율은 모두 레이블 1(양성)을 기준으로 한 값입니다.

2.2 학습·검증·테스트 데이터 분리

```
X_features = cancer_df.iloc[:, :-1]
y_label = cancer_df.iloc[:, -1]

# 전체 데이터 중 80%는 학습용 데이터, 20%는 테스트용 데이터 추출
X_train, X_test, y_train, y_test=train_test_split(X_features, y_label,
                                                  test_size=0.2, random_state=156 )

# 위에서 만든 X_train, y_train을 다시 쪼개서 90%는 학습과 10%는 검증용 데이터로 분리
X_tr, X_val, y_tr, y_val= train_test_split(X_train, y_train, test_size=0.1, random_state=156 )
print(X_train.shape , X_test.shape)
print(X_tr.shape, X_val.shape)
```

줄	코드	상세 설명
1~2	X_features = cancer_df.iloc[:, :-1] / y_label = cancer_df.iloc[:, -1]	맨 마지막 컬럼을 제외한 전부를 피쳐로, 맨 마지막 컬럼을 레이블로 분리합니다. 2.5장에서 배운 관용구입니다.
5~6	train_test_split(..., test_size=0.2)	569건 → 학습 455건 / 테스트 114건으로 1차 분리합니다.
9	X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.1)	매우 중요한 2차 분리입니다. 학습 데이터 455건을 다시 학습 409건 / 검증 46건으로 나눕니다. 검증 세트는 조기 중단 기준으로 사용됩니다.
10~11	print(...shape...)	분리 결과를 확인합니다.

▶ 실행 결과

```
(455, 30) (114, 30)
(409, 30) (46, 30)
```

■ 왜 검증 세트를 따로 만드는가

조기 중단은 "성능이 더 이상 좋아지지 않는 시점"을 판단해야 하는데, 그 판단에 **테스트 세트를 쓰면 안 됩니다**. 테스트 세트를 보고 학습을 멈추면 이미 테스트 정보가 모델에 반영된 것이므로 **성능이 과대평가**됩니다. 그래서 학습 데이터에서 **별도의 검증 세트를 떼어 내는 것**이 올바른 방법입니다.

2.3 DMatrix 생성

```
# 만약 구버전 XGBoost에서 DataFrame으로 DMatrix 생성이 안될 경우 X_train.values로 넘파이 변환.
# 학습, 검증, 테스트용 DMatrix를 생성.
dtr = xgb.DMatrix(data=X_tr, label=y_tr)
dval = xgb.DMatrix(data=X_val, label=y_val)
dtest = xgb.DMatrix(data=X_test, label=y_test)
```

줄	코드	상세 설명
3	dtr = xgb.DMatrix(data=X_tr, label=y_tr)	DMatrix 는 파이썬 래퍼 XGBoost 전용 데이터 구조입니다. 피쳐와 레이블을 하나의 객체에 함께 담습니다. 내부적으로 메모리 효율과 학습 속도에 최적화된 형태로 변환됩니다.
4~5	dval / dtest	검증용·테스트용도 동일하게 DMatrix로 변환합니다. 네이티브 API는 DMatrix만 입력받으므로 이 변환은 필수입니다.

■ 사이킷런 래퍼와의 결정적 차이

사이킷런 래퍼(`XGBClassifier`)는 **DMatrix 변환이 불필요**하며 DataFrame이나 ndarray를 그대로 받습니다. 이 번거로움이 사이킷런 래퍼를 선호하는 이유 중 하나입니다.

2.4 하이퍼 파라미터 설정

```
params = { 'max_depth':3,
           'eta': 0.05,
           'objective':'binary:logistic',
           'eval_metric':'logloss'
         }
num_rounds = 400
```

줄	코드	상세 설명
1	'max_depth':3	트리의 최대 깊이 입니다. 값이 클수록 모델이 복잡해지고 과적합 위험이 커집니다. 부스팅 계열은 얕은 트리(3~8) 를 많이 쌓는 것이 일반적입니다.
2	'eta': 0.05	학습률(learning rate) 입니다. 각 트리가 결과에 기여하는 정도를 조절하며, 작을수록 조금씩 신중하게 학습해 성능이 좋아지지만 더 많은 트리가 필요합니다. 사이킷런 래퍼에서는 <code>learning_rate</code> 라는 이름을 씁니다.
3	'objective':'binary:logistic'	목적 함수 입니다. <code>binary:logistic</code> 은 이진 분류 를 의미하며 예측 결과가 확률로 나옵니다. 다중 분류는 <code>multi:softmax</code> , 회귀는 <code>reg:squarederror</code> 를 씁니다.

4	'eval_metric': 'logloss'	평가 지표입니다. logloss 는 예측 확률이 실제와 얼마나 차이 나는지를 측정하며 작을수록 좋습니다 . 조기 중단은 이 값의 개선 여부로 판단합니다.
6	num_rounds = 400	부스팅 반복 횟수 (생성할 트리 개수)입니다. 조기 중단이 걸리면 400회를 다 채우지 않고 멈춥니다.

2.5 train()으로 학습과 조기 중단

```
# 학습 데이터 셋은 'train' 또는 평가 데이터 셋은 'eval' 로 명기합니다.
eval_list = [(dtr,'train'),(dval,'eval')] # 또는 eval_list = [(dval,'eval')] 만 명기해도 무방.
```

```
# 하이퍼 파라미터와 early stopping 파라미터를 train() 함수의 파라미터로 전달
xgb_model = xgb.train(params = params , dtrain=dtr , num_boost_round=num_rounds , \
                      early_stopping_rounds=50, evals=eval_list )
```

줄	코드	상세 설명
2	eval_list = [(dtr,'train'),(dval,'eval')]	학습 과정에서 성능을 모니터링할 데이터 목록 입니다. (DMatrix, 표시이름) 튜플의 리스트로 전달합니다. 이름은 출력 로그에 표시됩니다.
5~6	xgb.train(params=params, dtrain=dtr, num_boost_round=400, early_stopping_rounds=50, evals=eval_list)	네이티브 API의 학습 함수 입니다. params : 하이퍼 파라미터 딕셔너리 dtrain : 학습용 DMatrix num_boost_round : 최대 반복 횟수 early_stopping_rounds=50 : 50회 연속으로 평가 지표가 개선되지 않으면 중단 evals : 모니터링 데이터 목록 조기 중단을 쓰려면 evals 가 반드시 있어야 합니다. 기준으로 삼을 데이터가 없으면 개선 여부를 판단할 수 없기 때문입니다.

▶ 실행 결과 (일부 발췌)

```
[0] train-logloss:0.62473 eval-logloss:0.63088
[1] train-logloss:0.58662 eval-logloss:0.60461
[2] train-logloss:0.55210 eval-logloss:0.58205
...
[100] train-logloss:0.02802 eval-logloss:0.25385
...
[199] train-logloss:0.01101 eval-logloss:0.23824
[200] train-logloss:0.01096 eval-logloss:0.23802
[201] train-logloss:0.01090 eval-logloss:0.23814
[202] train-logloss:0.01086 eval-logloss:0.23762
[203] train-logloss:0.01080 eval-logloss:0.23740 ← 최저점(best)
[204] train-logloss:0.01074 eval-logloss:0.23752
...
[251] train-logloss:0.00925 eval-logloss:0.24046
[252] train-logloss:0.00921 eval-logloss:0.24055
[253] train-logloss:0.00919 eval-logloss:0.24053
```

```
best_iteration: 203
```

■ 결과 해석 - 조기 중단이 작동한 과정

- 203번째에서 eval-logloss가 **0.23740**으로 최저를 기록했습니다.
- 이후 50회 동안(204~253) 이 값을 넘어서는 개선이 없었으므로, **253번에서 학습이 중단**되었습니다. 400회를 모두 돌지 않고 약 **37%의 시간을 절약**한 것입니다.
- **train-logloss**는 계속 감소(0.0108 → 0.0092)하는데 **eval-logloss**는 오히려 증가(0.2374 → 0.2405)합니다. 이것이 과적합이 시작된 명확한 신호이며, 조기 중단이 이를 막아 준 것입니다.

⚠ early_stopping_rounds를 너무 작게 잡으면

값이 지나치게 작으면(예: 5) 성능이 일시적으로 정체되는 구간에서 성급하게 학습을 멈춰 **최적점에 도달하기 전에 종료**될 수 있습니다. 보통 전체 반복 횟수의 **10% 내외**로 설정합니다.

2.6 predict()와 예측 확률

```
pred_probs = xgb_model.predict(dtest)
print('predict( ) 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨')
print(np.round(pred_probs[:10],3))

# 예측 확률이 0.5 보다 크면 1 , 그렇지 않으면 0 으로 예측값 결정하여 List 객체인 preds에 저장
preds = [ 1 if x > 0.5 else 0 for x in pred_probs ]
print('예측값 10개만 표시:',preds[:10])
```

줄	코드	상세 설명
1	pred_probs = xgb_model.predict(dtest)	매우 중요한 차이점 입니다. 네이티브 API의 <code>predict()</code> 는 사이킷런과 달리 예측 레이블이 아니라 예측 확률 을 반환합니다. 입력도 DMatrix여야 합니다.
3	np.round(pred_probs[:10],3)	상위 10개 확률을 소수점 3자리로 출력합니다. 0.938, 0.004처럼 0~1 사이의 값 임을 확인할 수 있습니다.
6	preds = [1 if x > 0.5 else 0 for x in pred_probs]	리스트 컴프리헨션 으로 확률을 레이블로 변환합니다. 임계값 0.5 를 기준으로 크면 1, 아니면 0입니다. 네이티브 API를 쓸 때는 이 변환을 직접 해야 합니다.
7	print('예측값 10개만 표시:',preds[:10])	변환된 레이블을 확인합니다. 0.938 → 1, 0.004 → 0으로 잘 변환되었습니다.

▶ 실행 결과

```
predict( ) 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨
[0.938 0.004 0.776 0.058 0.975 1.      0.999 0.999 0.998 0.    ]
예측값 10개만 표시: [1, 0, 1, 0, 1, 1, 1, 1, 1, 0]
```

■ 임계값 0.5는 절대적이지 않다

암 진단처럼 **놓치면 안 되는 경우(재현율 중요)**에는 임계값을 낮춰 더 적극적으로 양성 판정을 하고, **오탐이 치명적인 경우(정밀도 중요)**에는 임계값을 높입니다. 임계값 조정은 네이티브 API에서 특히 직관적으로 다룰 수 있습니다.

2.7 평가 함수와 성능 측정

```
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score
from sklearn.metrics import f1_score, roc_auc_score
```

```
def get_clf_eval(y_test, pred=None, pred_proba=None):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    # ROC-AUC 추가
    roc_auc = roc_auc_score(y_test, pred_proba)
    print('오차 행렬')
    print(confusion)
    # ROC-AUC print 추가
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f},\
F1: {3:.4f}, AUC:{4:.4f}'.format(accuracy, precision, recall, f1, roc_auc))
```

줄	코드	상세 설명
5	def get_clf_eval(y_test, pred=None, pred_proba=None):	여러 평가 지표를 한 번에 출력하는 함수 입니다. 앞으로 여러 모델을 비교할 때 반복 호출하기 위해 함수로 만들었습니다.
6	confusion_matrix(y_test, pred)	오차 행렬 입니다. <code>[[TN, FP], [FN, TP]]</code> 형태의 2×2 배열로, 어떤 유형의 오류가 발생했는지 보여 줍니다.
7	accuracy_score	정확도 = (TN+TP) / 전체. 전체 중 맞힌 비율입니다.
8	precision_score	정밀도 = TP / (FP+TP). 1이라고 예측한 것 중 실제 1인 비율입니다.
9	recall_score	재현율 = TP / (FN+TP). 실제 1인 것 중 1로 맞힌 비율입니다.
10	f1_score	F1 스코어 . 정밀도와 재현율의 조화 평균 으로, 두 지표가 균형을 이룰 때 높아 집니다.
12	roc_auc_score(y_test, pred_proba)	ROC-AUC 입니다. 다른 지표와 달리 레이블이 아니라 예측 확률 을 인자로 받는다는 점에 반드시 주의해야 합니다. 1에 가까울수록 우수합니다.
16~17	print('정확도: {0:.4f}, ...')	줄 끝의 <code>\</code> 는 문자열을 다음 줄로 이어 쓰기 위한 연결 문자입니다.

```
get_clf_eval(y_test, preds, pred_probs)
```

▶ 실행 결과

```
오차 행렬
[[35  2]
 [ 2 75]]
정확도: 0.9649, 정밀도: 0.9740, 재현율: 0.9740, F1: 0.9740, AUC:0.9965
```

■ 오차 행렬 읽는 법

예측 0 (악성)		예측 1 (양성)
실제 0 (악성)	TN = 35 (악성을 악성으로 정확히 판정)	FP = 2 (악성인데 양성으로 오판 — 위험!)
실제 1 (양성)	FN = 2 (양성인데 악성으로 오판)	TP = 75 (양성을 양성으로 정확히 판정)

114건 중 **110건을 맞히고 4건을 틀렸습니다**(정확도 96.49%). 의료 관점에서는 **FP=2**, 즉 **실제 악성 종양을 양성으로 잘못 판정한 2건**이 가장 위험한 오류입니다. AUC 0.9965는 **매우 우수한 판별 능력**을 의미합니다.

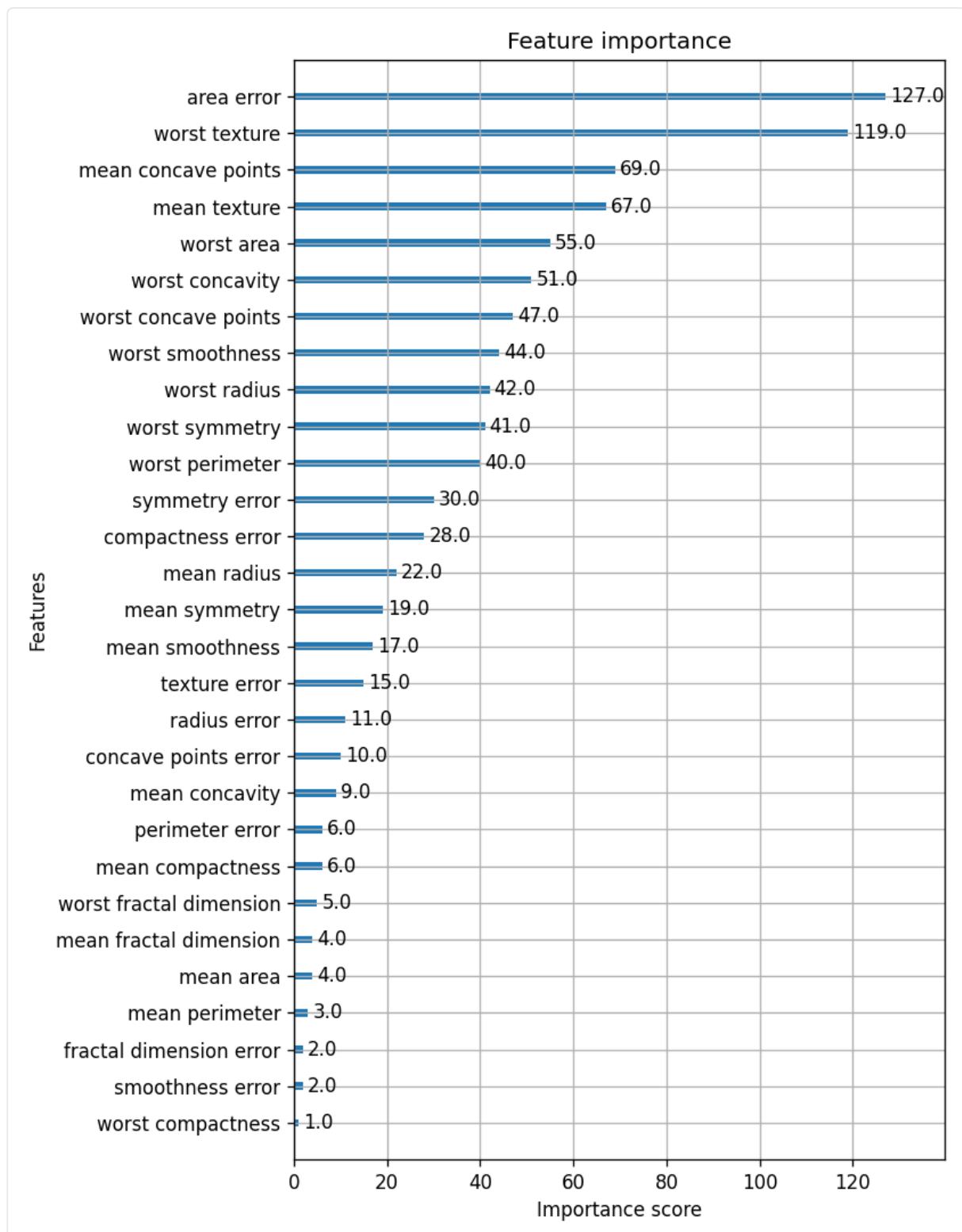
2.8 피쳐 중요도 시각화

```
import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(figsize=(10, 12))
plot_importance(xgb_model, ax=ax)
```

줄	코드	상세 설명
4	<code>fig, ax = plt.subplots(figsize=(10, 12))</code>	그래프 영역을 생성합니다. 피쳐가 30개라 세로로 길게(12인치) 잡았습니다. <code>subplots()</code> 는 Figure(전체 그림) 와 Axes(개별 그래프 축) 를 함께 반환합니다.
5	<code>plot_importance(xgb_model, ax=ax)</code>	XGBoost 자체 시각화 함수입니다. 학습된 모델을 넣으면 중요도 순으로 정렬된 가로 막대 그래프 를 그려 줍니다. <code>ax</code> 를 지정해 앞서 만든 영역에 그리게 합니다.

▶ 실행 결과



XGBoost 피쳐 중요도 (F score 기준)

■ F score의 의미

x축의 **F score**는 **F1 스코어가 아닙니다**. 해당 피쳐가 트리에서 분할 기준으로 사용된 횟수를 의미합니다(기본값 `importance_type='weight'`). 자주 쓰인 피쳐일수록 중요하다고 보는 방식입니다.

다른 기준도 지정할 수 있습니다: `'gain'` (분할로 얻은 평균 이득 — 실무에서 더 신뢰), `'cover'` (영향받은 샘플 수). 예: `plot_importance(xgb_model, ax=ax, importance_type='gain')`

3. 사이킷런 래퍼 XGBoost

3.1 XGBClassifier 기본 학습

```
# 사이킷런 래퍼 XGBoost 클래스인 XGBClassifier 임포트
from xgboost import XGBClassifier

xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.1, max_depth=3)
xgb_wrapper.fit(X_train, y_train)
w_preds = xgb_wrapper.predict(X_test)
w_pred_proba = xgb_wrapper.predict_proba(X_test)[: , 1]
```

줄	코드	상세 설명
4	XGBClassifier(n_estimators=400, learning_rate=0.1, max_depth=3)	파라미터를 딕셔너리가 아닌 생성자 인자 로 전달합니다. 이름도 사이킷런 관례를 따라 eta 대신 learning_rate , num_boost_round 대신 n_estimators 를 씁니다.
5	xgb_wrapper.fit(X_train, y_train)	DMatrix 변환 없이 DataFrame을 그대로 넣습니다. 다른 사이킷런 모델과 완전히 동일한 사용법입니다.
6	w_preds = xgb_wrapper.predict(X_test)	사이킷런 규약대로 예측 레이블(0/1) 이 바로 반환됩니다. 임계값 변환이 필요 없습니다.
7	w_pred_proba = xgb_wrapper.predict_proba(X_test)[: , 1]	확률은 predict_proba() 로 얻습니다. 반환은 (샘플수, 2) 2차원 배열 이며, [: , 1] 로 클래스 1일 확률만 추출합니다. ROC-AUC 계산에 필요합니다.

▶ 실행 결과 (get_clf_eval 호출)

```
오차 행렬
[[35  2]
 [ 1 76]]
정확도: 0.9737, 정밀도: 0.9744, 재현율: 0.9870,   F1: 0.9806, AUC:0.9951
```

■ 결과 해석

네이티브 API(0.9649)보다 **정확도가 높습니다(0.9737)**. 다만 이는 알고리즘의 우열이 아니라 **하이퍼 파라미터가 다르기 때문**입니다(학습률 0.05 vs 0.1, 조기 중단 유무). 두 API는 **동일한 조건이면 같은 결과**를 냅니다.

3.2 조기 중단 적용

```
from xgboost import XGBClassifier

xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.1, max_depth=3)
evals = [(X_test, y_test)]
xgb_wrapper.fit(X_train, y_train, early_stopping_rounds=100, eval_metric="logloss",
                eval_set=evals, verbose=True)

ws100_preds = xgb_wrapper.predict(X_test)
ws100_pred_proba = xgb_wrapper.predict_proba(X_test)[: , 1]
```

⚠ 이 코드는 최신 XGBoost에서 오류가 발생합니다

XGBoost 2.0부터 `fit()` 의 `early_stopping_rounds` 와 `eval_metric` 인자가 제거되었습니다. 위 코드를 XGBoost 3.2.0에서 실행하면 다음 오류가 납니다.

```
TypeError: XGBClassifier.fit() got an unexpected keyword argument 'early_stopping_rounds'
```

안정화 버전 기준 올바른 코드는 두 파라미터를 생성자로 옮기는 것입니다.

```
xgb_wrapper = XGBClassifier(n_estimators=400, learning_rate=0.1, max_depth=3,
                             early_stopping_rounds=100, eval_metric="logloss")
evals = [(X_test, y_test)]
xgb_wrapper.fit(X_train, y_train, eval_set=evals, verbose=True)
```

줄	코드	상세 설명
4	<code>evals = [(X_test, y_test)]</code>	조기 중단 기준이 될 데이터입니다. DMatrix가 아니라 원본 데이터를 튜플 로 넣습니다.
5	<code>early_stopping_rounds=100</code>	100회 연속 개선이 없으면 중단합니다.
5	<code>eval_metric="logloss"</code>	조기 중단 판단에 쓸 지표입니다.
6	<code>verbose=True</code>	매 반복의 평가 결과를 화면에 출력 합니다. False로 하면 조용히 학습합니다.

▶ 실행 결과 (수정된 코드 기준)

```
best_iteration: 105
```

```
오차 행렬
```

```
[[34  3]
```

```
 [ 1 76]]
```

```
정확도: 0.9649, 정밀도: 0.9620, 재현율: 0.9870,   F1: 0.9744, AUC:0.9951
```

⚠ 이 예제의 방법론적 문제점

이 코드는 `eval_set` 에 테스트 데이터(`X_test, y_test`)를 사용하고 있습니다. 앞서 설명했듯 테스트 데이터로 조기 중단 시점을 정하면 성능이 과대평가됩니다. 교재에서는 설명의 편의를 위해 이렇게 작성했지만, 실무에서는 반드시 별도의 검증 세트(`X_val, y_val`)를 사용해야 합니다. 앞의 네이티브 API 예제가 올바른 방식입니다.

3.3 early_stopping_rounds 값에 따른 차이

```
# early_stopping_rounds를 10으로 설정하고 재 학습.
xgb_wrapper.fit(X_train, y_train, early_stopping_rounds=10,
                 eval_metric="logloss", eval_set=evals, verbose=True)

ws10_preds = xgb_wrapper.predict(X_test)
ws10_pred_proba = xgb_wrapper.predict_proba(X_test[:, 1])
get_clf_eval(y_test, ws10_preds, ws10_pred_proba)
```

줄	코드	상세 설명
2	<code>early_stopping_rounds=10</code>	앞선 100회에서 10회로 크게 줄었습니다 . 성능 정체를 훨씬 성급하게 판단하게 됩니다. (최신 버전에서는 이 역시 생성자로 옮겨야 합니다.)

▶ 실행 결과

best_iteration: 70

오차 행렬

[[34 3]

[2 75]]

정확도: 0.9561, 정밀도: 0.9615, 재현율: 0.9740, F1: 0.9677, AUC:0.9954

■ 결과 해석 - 조기 중단 값의 트레이드오프

early_stopping_rounds	best_iteration	정확도	해석
100	105	0.9649	충분히 탐색한 뒤 중단
10	70	0.9561	너무 일찍 중단되어 성능 하락

10으로 줄이자 70번째에서 조기 종료되어 정확도가 1건만큼 떨어졌습니다. 이는 학습이 일시적인 정체 구간을 지나 더 좋아질 수 있었는데 그 전에 멈췄기 때문입니다. 조기 중단 값은 너무 크면 시간 낭비, 너무 작으면 성능 손실이라는 트레이드오프가 있습니다.

3.4 피쳐 중요도

```
from xgboost import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(figsize=(10, 12))
# 사이킷런 래퍼 클래스를 입력해도 무방.
plot_importance(xgb_wrapper, ax=ax)
```

줄	코드	상세 설명
7	plot_importance(xgb_wrapper, ax=ax)	plot_importance() 는 네이티브 모델과 사이킷런 래퍼 모두 입력으로 받을 수 있습니다. 사용법이 완전히 동일합니다.

■ 사이킷런 방식의 중요도

xgb_wrapper.feature_importances_ 속성으로도 중요도를 얻을 수 있습니다. 이는 사이킷런의 다른 트리 모델과 동일한 인터페이스이므로, seaborn 등으로 자유롭게 시각화할 수 있습니다.

4. 핵심 요약 및 머신러닝 활용 포인트

4.1 두 API 비교 요약

항목	파이썬 래퍼(네이티브)	사이킷런 래퍼
임포트	import xgboost as xgb	from xgboost import XGBClassifier
데이터	xgb.DMatrix(data=X, label=y)	DataFrame/ndarray 그대로
파라미터	딕셔너리 params	생성자 인자

학습	<code>xgb.train(params, dtrain, ...)</code>	<code>model.fit(X, y)</code>
학습률	<code>eta</code>	<code>learning_rate</code>
반복 횟수	<code>num_boost_round</code>	<code>n_estimators</code>
예측	<code>predict()</code> → 확률	<code>predict()</code> → 레이블 <code>predict_proba()</code> → 확률
모니터링	<code>evals=[(dmatrix, '이름')]</code>	<code>eval_set=[(X, y)]</code>

4.2 주요 하이퍼 파라미터

파라미터	역할	일반적 범위 / 튜닝 방향
<code>n_estimators</code> / <code>num_boost_round</code>	트리 개수(반복 횟수)	크게 잡고 조기 중단으로 제어
<code>learning_rate</code> / <code>eta</code>	학습률	0.01~0.2. 낮추면 트리 수를 늘려야 함
<code>max_depth</code>	트리 최대 깊이	3~10. 과적합 시 낮춤
<code>min_child_weight</code>	자식 노드 최소 가중치 합	과적합 시 높임
<code>subsample</code>	트리별 데이터 샘플링 비율	0.5~1. 과적합 시 낮춤
<code>colsample_bytree</code>	트리별 피쳐 샘플링 비율	0.5~1. 과적합 시 낮춤
<code>reg_lambda</code> / <code>reg_alpha</code>	L2 / L1 규제	과적합 시 높임
<code>scale_pos_weight</code>	클래스 불균형 보정	음성/양성 비율값 지정

4.3 머신러닝 활용 포인트

- 사이킷런 래퍼를 기본으로 : `GridSearchCV`, `cross_val_score`, `Pipeline` 과 그대로 연동되어 워크플로가 훨씬 간결해집니다.
- 조기 중단은 필수 기법 : `n_estimators` 를 크게 잡고 조기 중단으로 최적 지점을 자동으로 찾는 것이 표준 전략입니다.
- 검증 세트를 반드시 분리 : 조기 중단 기준으로 테스트 세트를 쓰면 성능이 부풀려집니다. 학습 데이터를 다시 나눠 검증 세트를 만드세요.
- 과적합 신호 읽기 : train 지표는 계속 좋아지는데 eval 지표가 나빠지기 시작하는 지점이 과적합의 시작입니다.
- 피쳐 중요도는 **gain** 기준으로 : 기본값 `weight`(사용 횟수)보다 `importance_type='gain'` 이 실제 기여도를 더 잘 반영합니다.
- 다음 단계 : XGBoost보다 더 빠르고 메모리 효율이 좋은 **LightGBM**(4.7장)과, 하이퍼 파라미터를 자동 탐색하는 **HyperOpt**(4.8장)를 이어서 학습하세요.

⚠ 안정화 버전 참고 (XGBoost 3.2.0 기준) — 반드시 확인

- 가장 중요한 변경 : `fit( )` 의 `early_stopping_rounds`, `eval_metric` 인자가 **2.0에서 제거**되었습니다. 반드시 **생성자 인자** 로 전달하세요. 구버전 교재 코드는 `TypeError` 가 발생합니다.
- 네이티브 `xgb.train( )` 의 `early_stopping_rounds` 는 그대로 사용 가능합니다.
- 1.6 이전에는 `use_label_encoder=False` 가 필요했으나 현재는 **불필요**합니다.
- `best_ntree_limit` 속성은 제거되었고 `best_iteration` 을 사용합니다.
- GPU 사용 시 `tree_method='gpu_hist'` 대신 `device='cuda'` 를 사용합니다.